CI/CD Integration

> **Series:** AUTOM | **Notebook:** 7 of 8 | **Created:** January 2026 | **Last Updated:** 02/14/2026

CI/CD integration brings software development practices to Dynatrace configuration management. By storing configs in Git and deploying via pipelines, teams gain version control, review processes, and automated deployments.

Table of Contents

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
CI/CD Platform	GitHub Actions, GitLab CI, or Jenkins
Monaco or Terraform	One of the config-as-code tools
Git Repository	For storing configurations
Authentication	API Token + Platform Token for full coverage (see AUTOM-04)
HashiCorp Vault	*Optional* — for runtime credential retrieval instead of static CI/CD secrets
OPA / Conftest	*Optional* — for policy-as-code gates in pipelines

Token Types Reminder

As of Dynatrace Terraform provider **v1.88.0**, synthetic monitors and SLOs require a classic **API Token** (`dt0c01`). For full resource coverage in pipelines, use **Platform Token + API Token** together. See AUTOM-04: Provider Configuration for details.

Learning Objectives

By the end of this notebook, you will:

- Understand GitOps principles for Dynatrace
- Know how to set up CI/CD pipelines for config deployment
- Be able to implement pull request workflows with plan-as-PR-comment
- Handle multi-environment deployments with combined auth
- Integrate HashiCorp Vault for runtime credential retrieval
- Implement policy-as-code gates with OPA/Conftest and Sentinel
- Set up automated drift detection workflows
- Create reusable GitHub Actions workflows for multi-team organizations

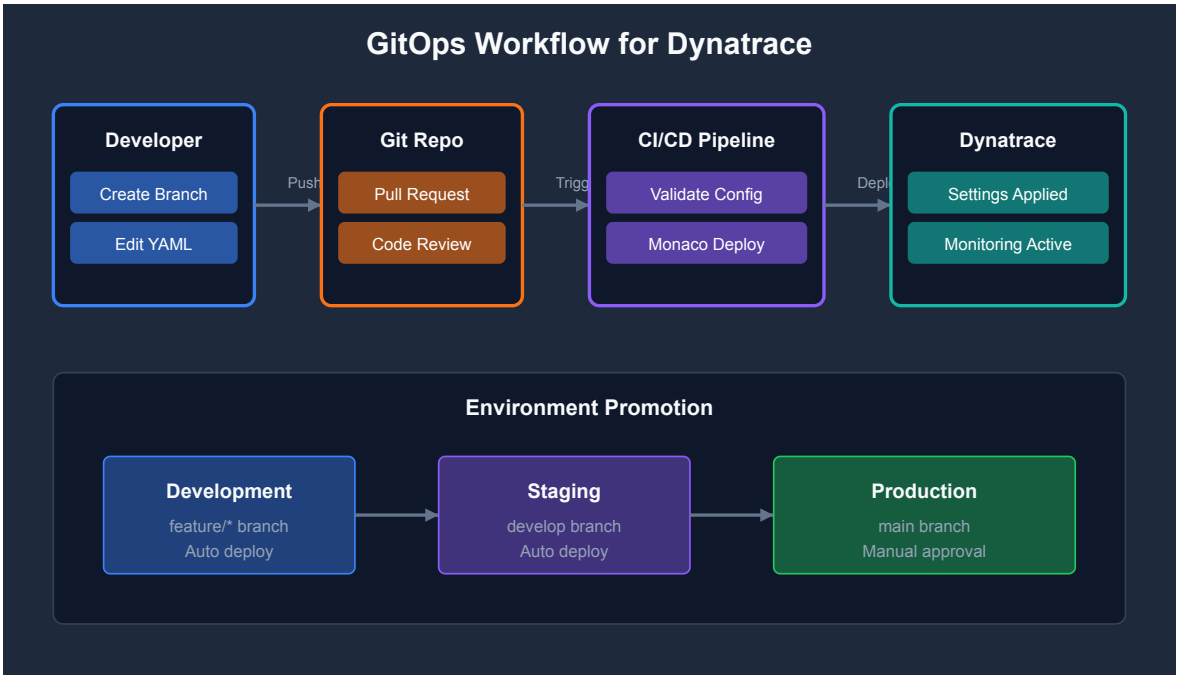
---

1. Introduction

Why CI/CD for Dynatrace?

Benefit	Description
Version Control	All changes tracked in Git history
Code Review	Pull requests for config changes
Automation	No manual deployments
Rollback	Revert to any previous state
Audit Trail	Who changed what, when

GitOps Workflow



2. GitOps Fundamentals

Repository Structure

```
dynatrace-config/
├── .github/
│   └── workflows/
│       ├── validate.yml
│       └── deploy.yml
├── manifest.yaml          # Monaco manifest
├── environments/
│   ├── dev.yaml
│   ├── staging.yaml
│   └── production.yaml
├── projects/
│   ├── alerting/
│   ├── management-zones/
│   └── synthetic/
└── README.md
```

Branch Strategy

Branch	Purpose	Deploys To
`main`	Production configs	Production
`develop`	Integration branch	Staging
`feature/*`	New features	Dev (optional)

Environment Promotion

```
feature/* → develop → main
    ↓          ↓        ↓
   Dev     Staging   Production
```

3. GitHub Actions

Validation Workflow

.github/workflows/validate.yml:

```
name: Validate Dynatrace Config

on:
  pull_request:
    branches: [main, develop]
    paths:
      - 'projects/**'
      - 'manifest.yaml'

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Validate Configuration
        run: monaco validate manifest.yaml

      - name: Dry Run Deploy
        env:
          DT_DEV_URL: ${{ secrets.DT_DEV_URL }}
          DT_DEV_TOKEN: ${{ secrets.DT_DEV_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment development --dry-run
```

Deployment Workflow

.github/workflows/deploy.yml:

```yaml
name: Deploy Dynatrace Config

on:
  push:
    branches: [main]
    paths:
      - 'projects/**'
      - 'manifest.yaml'

jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Staging
        env:
          DT_STAGING_URL: ${{ secrets.DT_STAGING_URL }}
          DT_STAGING_TOKEN: ${{ secrets.DT_STAGING_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment staging

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Production
        env:
          DT_PROD_URL: ${{ secrets.DT_PROD_URL }}
```

```
        DT_PROD_TOKEN: ${{ secrets.DT_PROD_TOKEN }}
      run: |
        monaco deploy manifest.yaml --environment production
```

Terraform Workflow with Combined Auth

For Terraform-based deployments, use **Platform Token + API Token** together for full resource coverage. The provider automatically routes each resource to the correct token.

```
name: Terraform Dynatrace

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.6.0

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        id: plan
        if: github.event_name == 'pull_request'
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Post Plan to PR
        if: github.event_name == 'pull_request'
        uses: borchero/terraform-plan-comment@v2
        with:
          token: ${{ github.token }}
          planfile: tfplan

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name ==
'push'
        run: terraform apply -auto-approve
        env:
```

```
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Secret	Token Type	Covers
`DT_PLATFORM_TOKEN`	`dt0s16.xxxx`	Settings 2.0 + Gen3 Platform (workflows, documents, segments)
`DT_API_TOKEN`	`dt0c01.xxxx`	Synthetic monitors, SLOs (v1.88.0 requirement)
`DT_ENV_URL`	—	Tenant URL

> **Why combined auth?** A single API Token cannot manage Gen3 resources. A single Platform Token cannot manage synthetics/SLOs (removed in v1.88.0). Using both together gives the pipeline full coverage.

Vault Integration for Runtime Credentials

For enterprise environments with **short-lived tokens** (e.g., 24-hour expiration), retrieve credentials at runtime from **HashiCorp Vault** instead of storing them as static CI/CD secrets.

```
name: Terraform with Vault Credentials

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      id-token: write       # Required for OIDC → Vault JWT auth
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4

      - name: Retrieve Dynatrace credentials from Vault
        uses: hashicorp/vault-action@v3
        with:
          url: ${{ secrets.VAULT_ADDR }}
          method: jwt
          role: dynatrace-deployer
          secrets: |
            secret/data/dynatrace/prod platform_token |
DYNATRACE_PLATFORM_TOKEN ;
            secret/data/dynatrace/prod api_token | DYNATRACE_API_TOKEN ;
            secret/data/dynatrace/prod env_url | DYNATRACE_ENV_URL

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' &amp;amp;&amp;amp; github.event_name ==
'push'
        run: terraform apply -auto-approve
        env:
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Vault path structure per team per tenant:

```
secret/
└── dynatrace/
    ├── dev/
    │   ├── platform_token    # dt0s16 for dev tenant
    │   ├── api_token         # dt0c01 for dev tenant
    │   └── env_url           # https://dev.live.dynatrace.com
    ├── prod/
    │   ├── platform_token
    │   ├── api_token
    │   └── env_url
    └── teams/
        ├── payments/         # Team-specific OAuth clients
        │   ├── client_id
        │   └── client_secret
        └── platform/
            ├── client_id
            └── client_secret
```

> **Why Vault?** Static GitHub Secrets don't expire. If your organization uses 24-hour token rotation or needs audit trails for credential access, Vault provides runtime retrieval with automatic expiration and access logging.

Policy-as-Code Gates (OPA / Sentinel)

Add governance checks to your pipeline using **OPA/Conftest** (open-source) or **Sentinel** (Terraform Enterprise/Cloud).

OPA/Conftest — Resource Type Allowlist

Restrict which Dynatrace resource types a team can create:

policy/allowed_resources.rego:

```
package main

# Only allow these Dynatrace resource types
allowed_resources := {
  "dynatrace_alerting",
  "dynatrace_autotag_v2",
  "dynatrace_management_zone_v2",
  "dynatrace_maintenance"
}

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  startswith(resource.type, "dynatrace_")
  not allowed_resources[resource.type]
  msg := sprintf("Resource type '%s' is not in the allowlist",
[resource.type])
}
```

OPA/Conftest — Mandatory Team Tagging

Ensure all resources include team ownership metadata:

policy/mandatory_tags.rego:

```
package main

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  resource.type == "dynatrace_autotag_v2"
  not resource.values.name
  msg := "Auto-tag resources must have a name"
}

deny[msg] {
  resource := input.planned_values.root_module.resources[_]
  startswith(resource.type, "dynatrace_")
  not has_team_ownership(resource)
  msg := sprintf("Resource '%s' must include team ownership metadata",
[resource.address])
}

has_team_ownership(resource) {
  contains(resource.values.name, resource.values.team_name)
}
```

Pipeline Integration

```
# Add after terraform plan step
- name: Install Conftest
  run: |
    wget -q https://github.com/open-policy-
agent/conftest/releases/latest/download/conftest_Linux_x86_64.tar.gz
    tar xzf conftest_Linux_x86_64.tar.gz
    sudo mv conftest /usr/local/bin/

- name: Run Policy Checks
  run: |
    terraform show -json tfplan &gt; tfplan.json
    conftest test tfplan.json --policy policy/
```

Sentinel (Terraform Enterprise / Cloud)

For teams using **TFE**, Sentinel policies enforce governance at the workspace level:

```
# sentinel/restrict_resource_types.sentinel
import "tfplan/v2" as tfplan

allowed_types = [
  "dynatrace_alerting",
  "dynatrace_autotag_v2",
  "dynatrace_management_zone_v2",
]

main = rule {
  all tfplan.resource_changes as _, rc {
    rc.type in allowed_types or not (rc.type matches "dynatrace_.*")
  }
}
```

Drift Detection Workflow

Schedule automatic drift detection to catch manual UI changes ("ClickOps") that bypass your Terraform pipeline:

```
name: Drift Detection

on:
  schedule:
    - cron: '0 6 * * 1-5'   # Every weekday at 6am UTC
  workflow_dispatch: {}       # Allow manual trigger

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init

      - name: Check for Drift
        id: drift
        run: |
          set +e
          terraform plan -detailed-exitcode -no-color 2&gt;&amp;1 | tee
drift-output.txt
          echo "exitcode=$?" &gt;&gt; "$GITHUB_OUTPUT"
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Create Issue on Drift
        if: steps.drift.outputs.exitcode == '2'
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const drift = fs.readFileSync('drift-output.txt',
'utf8').slice(-3000);
            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: '⚠️ Dynatrace configuration drift detected',
              body: `Scheduled drift detection found changes not in Terraform
state.\n\n\`\`\`\n${drift}\n\`\`\`\n\nSomeone may have made manual changes in
the Dynatrace UI.`,
```

```
            labels: ['drift', 'dynatrace']
          });
```

> **Exit codes:** `terraform plan -detailed-exitcode` returns `0` = no changes, `1` = error, `2` = drift detected. This lets your pipeline take different actions based on the result.

Reusable GitHub Actions Workflows

For organizations managing multiple Dynatrace tenants or LOB repositories, create **reusable workflows** that any repo can call:

.github/workflows/terraform-dynatrace.yml (in a shared workflow repo):

```yaml
name: Terraform Dynatrace (Reusable)

on:
  workflow_call:
    inputs:
      working_directory:
        required: false
        type: string
        default: '.'
      terraform_version:
        required: false
        type: string
        default: '1.6.0'
    secrets:
      DT_ENV_URL:
        required: true
      DT_PLATFORM_TOKEN:
        required: true
      DT_API_TOKEN:
        required: true

jobs:
  plan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    defaults:
      run:
        working-directory: ${{ inputs.working_directory }}
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ inputs.terraform_version }}

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
```

```
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Post Plan to PR
        if: github.event_name == 'pull_request'
        uses: borchero/terraform-plan-comment@v2
        with:
          token: ${{ github.token }}
          planfile: tfplan

  apply:
    needs: plan
    if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name ==
'push'
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ${{ inputs.working_directory }}
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ inputs.terraform_version }}

      - name: Terraform Init
        run: terraform init

      - name: Terraform Apply
        run: terraform apply -auto-approve
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Calling the reusable workflow from a LOB repo:

```
# .github/workflows/deploy.yml (in dt-lob-payments repo)
name: Deploy Dynatrace Config

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    uses: my-org/dt-templates/.github/workflows/terraform-dynatrace.yml@main
    with:
      working_directory: terraform/
    secrets:
      DT_ENV_URL: ${{ secrets.DT_ENV_URL }}
      DT_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
      DT_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
```

> **Multi-tenant pattern:** For organizations with 5+ tenants, the reusable workflow is called once per environment with different secrets, providing a consistent deployment experience across all tenants.

---

4. GitLab CI/CD

Pipeline Configuration

.gitlab-ci.yml:

```
stages:
  - validate
  - deploy-staging
  - deploy-production

variables:
  MONACO_VERSION: "2.0.0"

.monaco-setup: &amp;monaco-setup
  before_script:
    - curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/download/v${MONACO_VERSION}/monaco-linux-amd64 -o monaco
    - chmod +x monaco
    - mv monaco /usr/local/bin/

validate:
  stage: validate
  &lt;&lt;: *monaco-setup
  script:
    - monaco validate manifest.yaml
    - monaco deploy manifest.yaml --environment development --dry-run
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

deploy-staging:
  stage: deploy-staging
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment staging
  environment:
    name: staging
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy-production:
  stage: deploy-production
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment production
  environment:
    name: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Variables Configuration

In GitLab: **Settings → CI/CD → Variables**

Variable	Protected	Masked
`DT_DEV_URL`	No	No
`DT_DEV_TOKEN`	No	Yes
`DT_STAGING_URL`	No	No
`DT_STAGING_TOKEN`	Yes	Yes
`DT_PROD_URL`	Yes	No
`DT_PROD_TOKEN`	Yes	Yes

---

5. ArgoCD Integration

For Kubernetes-native GitOps, use ArgoCD with Dynatrace configs.

ArgoCD Application for Monaco Configs

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-config
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/dynatrace-config.git
    targetRevision: HEAD
    path: projects
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

ArgoCD for Dynatrace Operator (DynaKube)

Deploy and manage the Dynatrace Operator via ArgoCD:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-operator
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/k8s-platform.git
    targetRevision: HEAD
    path: dynatrace
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Repository structure for Dynatrace Operator:

```
dynatrace/
├── kustomization.yaml
├── namespace.yaml
├── operator.yaml          # Dynatrace Operator deployment
├── dynakube.yaml          # DynaKube custom resource
└── secrets/
    └── dynakube-secret.yaml  # External Secrets reference
```

External Secrets for Token Management

Use External Secrets Operator to sync tokens from secret stores:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: dynakube-tokens
  namespace: dynatrace
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: dynakube
    creationPolicy: Owner
  data:
    - secretKey: apiToken
      remoteRef:
        key: dynatrace/api-token
    - secretKey: dataIngestToken
      remoteRef:
        key: dynatrace/data-ingest-token
```

Using Config Management Plugins

argocd-cm ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  configManagementPlugins: |
    - name: monaco
      generate:
        command: ["sh", "-c"]
        args: ["monaco deploy manifest.yaml --environment
$ARGOCD_ENV_ENVIRONMENT"]
```

---

6. FluxCD Integration

FluxCD provides an alternative GitOps approach with a pull-based reconciliation model.

FluxCD vs ArgoCD

Feature	FluxCD	ArgoCD
Architecture	Controller-based	Server + UI
UI	Minimal (Weave GitOps)	Built-in web UI
Multi-tenancy	Namespace-based	Project-based
Helm Support	Native HelmRelease	Application CRD
Image Automation	Built-in	Requires Argo CD Image Updater

FluxCD for Dynatrace Operator

GitRepository source:

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: dynatrace-config
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/org/k8s-platform
  ref:
    branch: main
  secretRef:
    name: github-token
```

Kustomization for Dynatrace:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 10m
  targetNamespace: dynatrace
  sourceRef:
    kind: GitRepository
    name: dynatrace-config
  path: ./dynatrace
  prune: true
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: dynatrace-operator
      namespace: dynatrace
```

FluxCD HelmRelease for Dynatrace Operator

Deploy via Helm with FluxCD:

```
apiVersion: source.toolkit.fluxcd.io/v1beta2
kind: HelmRepository
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 1h
  url: https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable

---
apiVersion: helm.toolkit.fluxcd.io/v2beta1
kind: HelmRelease
metadata:
  name: dynatrace-operator
  namespace: dynatrace
spec:
  interval: 5m
  chart:
    spec:
      chart: dynatrace-operator
      version: "&gt;=1.0.0"
      sourceRef:
        kind: HelmRepository
        name: dynatrace
        namespace: flux-system
  values:
    installCRD: true
```

SOPS for Secret Management with FluxCD

Use SOPS to encrypt secrets in Git:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace-secrets
  namespace: flux-system
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: dynatrace-config
  path: ./dynatrace/secrets
  prune: true
  decryption:
    provider: sops
    secretRef:
      name: sops-age
```

---

7. Dynatrace Operator GitOps Patterns

When deploying the Dynatrace Operator via GitOps, follow these patterns for production environments.

> **Important:** Use `apiVersion: dynatrace.com/v1beta5` for Dynatrace Operator 1.7.0+. Earlier versions (v1beta1, v1beta2) are deprecated and no longer supported.

Multi-Cluster Deployment Pattern

For organizations with multiple Kubernetes clusters:

```
platform-gitops/
├── clusters/
│   ├── production-east/
│   │   └── dynatrace/
│   │       ├── kustomization.yaml
│   │       └── dynakube-patch.yaml     # Cluster-specific overrides
│   ├── production-west/
│   │   └── dynatrace/
│   │       ├── kustomization.yaml
│   │       └── dynakube-patch.yaml
│   └── staging/
│       └── dynatrace/
│           ├── kustomization.yaml
│           └── dynakube-patch.yaml
└── base/
    └── dynatrace/
        ├── kustomization.yaml
        ├── namespace.yaml
        ├── operator.yaml
        └── dynakube.yaml               # Base DynaKube config
```

Base kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - namespace.yaml
  - https://github.com/Dynatrace/dynatrace-
operator/releases/download/v1.3.2/kubernetes.yaml
  - dynakube.yaml
```

Cluster overlay (production-east):

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base/dynatrace
patchesStrategicMerge:
  - dynakube-patch.yaml
```

Cluster-specific patch:

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  oneAgent:
    cloudNativeFullStack:
      args:
        - --set-host-group=production-east
```

Multi-Tenant Observability Pattern

For clusters serving multiple teams with different Dynatrace tenants:

```
# Team A - uses tenant-a.live.dynatrace.com
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: team-a-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://tenant-a.live.dynatrace.com/api
  tokens: team-a-tokens
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-a

---
# Team B - uses tenant-b.live.dynatrace.com
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: team-b-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://tenant-b.live.dynatrace.com/api
  tokens: team-b-tokens
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-b
```

GitOps Health Checks

Configure sync health checks to verify Dynatrace deployment:

ArgoCD health check:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace
spec:
  # ... source config ...
  ignoreDifferences:
    - group: dynatrace.com
      kind: DynaKube
      jsonPointers:
        - /status  # Ignore status field changes
```

FluxCD health check:

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace
spec:
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: dynatrace-operator
      namespace: dynatrace
    - apiVersion: apps/v1
      kind: DaemonSet
      name: dynakube-oneagent
      namespace: dynatrace
```

Sealed Secrets Alternative

For Kubernetes-native secret encryption:

```
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  encryptedData:
    apiToken: AgBy8BYo...encrypted...
    dataIngestToken: AgCtr4s2...encrypted...
```

Generate sealed secrets:

```
# Seal the secret (use your actual tokens from Dynatrace)
kubectl create secret generic dynakube \
  --from-literal=apiToken=&lt;your-api-token&gt; \
  --from-literal=dataIngestToken=&lt;your-data-ingest-token&gt; \
  --dry-run=client -o yaml | \
  kubeseal --format yaml &gt; sealed-dynakube.yaml
```

---

8. Best Practices

Security

Practice	Description
Masked secrets	Always mask API tokens in CI/CD
Protected branches	Require reviews for main branch
Environment protection	Manual approval for production
Token rotation	Rotate tokens regularly
Least privilege	Minimal token scopes
External Secrets	Use ESO, SOPS, or Sealed Secrets for K8s
Vault for runtime creds	Retrieve tokens at pipeline runtime instead of static CI/CD secrets
Combined auth	Use Platform Token + API Token for full resource coverage (v1.88.0)
Policy-as-code	OPA/Conftest or Sentinel to enforce resource allowlists and mandatory tagging

Workflow Design

Practice	Description
Validate first	Always validate before deploy
Dry run PRs	Show what would change
Plan as PR comment	Post `terraform plan` output directly on PRs for reviewer context
Staged rollout	Dev → Staging → Production
Manual gates	Require approval for production
Rollback plan	Know how to revert changes
Health checks	Verify deployment success
Drift detection	Schedule `terraform plan -detailed-exitcode` to catch ClickOps
Reusable workflows	`workflow_call` for consistent deployment across repos

GitOps Tool Selection

Use Case	Recommended Tool
Simple CI/CD	GitHub Actions / GitLab CI
K8s-native, UI preferred	ArgoCD
K8s-native, Helm-first	FluxCD
Multi-cluster enterprise	ArgoCD or FluxCD

Pull Request Template

.github/PULL_REQUEST_TEMPLATE.md:

```
&lt;a id="dynatrace-configuration-change"&gt;&lt;/a&gt;
## Dynatrace Configuration Change
### Description
&lt;!-- What configuration is being changed? --&gt;

### Type of Change
- [ ] New configuration
- [ ] Modification to existing config
- [ ] Deletion of config

### Environments Affected
- [ ] Development
- [ ] Staging
- [ ] Production

### Validation
- [ ] Monaco validate passed
- [ ] Dry run successful
- [ ] Tested in dev environment

### Rollback Plan
&lt;!-- How to revert if issues arise? --&gt;
```

Notification Integration

Add deployment notifications:

```
# GitHub Actions notification step
- name: Notify Slack
  if: always()
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {
        "text": "Dynatrace config deployment: ${{ job.status }}",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "*Deployment to ${{ github.ref_name }}*\nStatus: ${{
job.status }}\nCommit: ${{ github.sha }}"
            }
          }
        ]
      }
  env:
    SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK }}
```

---

9. Next Steps

Deployment Event Tracking

Send deployment events to Dynatrace:

```
- name: Send Deployment Event
  run: |
    curl -X POST "${{ secrets.DT_URL }}/api/v2/events/ingest" \
      -H "Authorization: Api-Token ${{ secrets.DT_TOKEN }}" \
      -H "Content-Type: application/json" \
      -d '{
        "eventType": "CUSTOM_DEPLOYMENT",
        "title": "Dynatrace Config Deployment",
        "properties": {
          "source": "github-actions",
          "commit": "${{ github.sha }}",
          "branch": "${{ github.ref_name }}"
        }
      }'
```

Continue the Series

Next Notebook	Focus
AUTOM-08: Migration Automation	Bulk configuration transfer between tenants

Additional Resources

- GitHub Actions Documentation
- GitLab CI/CD Documentation
- ArgoCD Documentation
- FluxCD Documentation
- Dynatrace Operator
- Monaco GitHub Repository
- External Secrets Operator
- OPA/Conftest
- Sentinel Documentation
- HashiCorp Vault Action

Summary

In this notebook, you learned:

- GitOps fundamentals for Dynatrace configuration
- Setting up GitHub Actions and GitLab CI/CD pipelines
- **Combined auth** (Platform Token + API Token) for full resource coverage in Terraform pipelines
- **Vault integration** for runtime credential retrieval with short-lived tokens
- **Policy-as-code gates** using OPA/Conftest and Sentinel for governance enforcement
- **Drift detection** workflows to catch manual UI changes
- **Reusable workflows** (`workflow_call`) for multi-team organizations
- ArgoCD integration with External Secrets for token management
- FluxCD with HelmRelease and SOPS for secret encryption
- Dynatrace Operator GitOps patterns for multi-cluster and multi-tenant environments

> **Key Takeaway:** CI/CD integration transforms Dynatrace config management into a software development workflow. For full resource coverage, use combined auth (Platform Token + API Token). For enterprise governance, add Vault for secrets, OPA/Sentinel for policy enforcement, and scheduled drift detection to catch ClickOps.

Continue to AUTOM-08: Migration Automation to learn bulk configuration transfer.
